

Banner Automation Testing (POC)

Introduction

Pharmaceutical companies run **digital banner advertisements** (300×250px, 728×90px, etc.) for prescription drugs like NUBEQA, LIVDELZI, Nplate — and every one of these banners must comply with **FDA regulations**. The FDA mandates that:

- The banner must visibly render without errors
- It must have a clear visual boundary (border) distinguishing it from surrounding page content
- Text must **not** be selectable/copyable (to prevent out-of-context quoting of medical claims)
- The banner must load within a reasonable time
- Any **Important Safety Information (ISI)** — drug warnings, adverse reactions, prescribing info — must be **fully readable**
- If ISI overflows the visible area, it **must auto-scroll** so every word is seen
- A user must be able to manually scroll the ISI with their mouse wheel

Previously this was checked manually by QA engineers. Banner Automation Testing automates the entire suite using a real browser (Playwright), DOM/CSS evidence collection, screenshots, and a Vision AI (Ollama local LLM) that acts as a senior QA reviewer.

The 7 Checks Intuition and Pass/Fail Criteria

CHECK 1 — Banner Visible (banner_visible) → RenderAgent

What it checks: Did the banner actually load and render something meaningful?

Intuition: A blank or error page is the most basic failure. The agent collects DOM readiness signals and counts visible elements.

Evidence collected:

- `readyState` — must be "complete" (page finished loading)
- `bodyWidth / bodyHeight` — banner dimensions in pixels (must be > 0)
- `visibleElements / totalElements` — how many DOM elements are actually rendered with non-zero size
- `imageCount / brokenImageCount` — images that failed to load (404s, bad URLs)
- `pageTitle` — human-readable sanity check

PASS criteria (ALL must be true):

- Screenshot shows recognisable banner content
- `bodyWidth > 0 AND bodyHeight > 0`
- `visibleElements > 0`
- Not a browser error page (`net::ERR_*`, "This site can't be reached")

FAIL criteria (ANY sufficient):

- Screenshot is completely blank
- `brokenImageCount == imageCount` (every single image is broken)
- `readyState != "complete" AND` page looks empty

Why this matters: If the banner doesn't render, everything else is irrelevant. Also catches CDN failures and broken asset paths.

CHECK 2 — Border Detected (`border_detected`) → VisualAgent

What it checks: Does the banner have a visible visual boundary separating it from the surrounding page?

Intuition: FDA guidelines require that digital pharma ads be clearly delineated from surrounding editorial content. Without a border, a user can't tell where the ad ends and the article begins — which is a regulatory violation.

Evidence collected:

- CSS scan across **all frames and iframes** for: border, outline, box-shadow

- Background colour check — a strongly coloured background also serves as a boundary
- `deterministicResult` — pre-computed: PASS if any border OR distinct background found, else `NEEDS_VISUAL_CHECK`
- Two screenshots: one with detected borders **highlighted in 4px red overlays**, one plain

PASS criteria:

- `deterministicResult` = PASS (CSS confirms at least one border element) → immediate pass, no further reasoning needed
- OR: `NEEDS_VISUAL_CHECK` and LLM sees a colour difference at banner edges in the plain screenshot

FAIL criteria:

- `deterministicResult` = `NEEDS_VISUAL_CHECK` AND screenshot shows banner edge blending invisibly into white page

Why the 2-screenshot strategy: The highlighted screenshot makes it visually obvious to the LLM which element has the border (red box drawn around it). The plain screenshot shows what a real user would actually see. Together they eliminate ambiguity.

CHECK 3 — Copy Selection Disabled (`copy_selection_disabled`) → `InteractionAgent`

What it checks: Can a user highlight and copy the banner's text?

Intuition: Pharma ad text contains precise medical language (dosages, indications, warnings). If a user could select and paste "NUBEQA reduces PSA by 86%" out of context, it could be misleading. Banners must set `user-select: none` to block this.

Evidence collected:

- `bodyUserSelect` / `htmlUserSelect` — the computed CSS `user-select` property on `<body>` and `<html>`
- `keyboardSelectedLength` — after simulating `Ctrl+A`, how many characters were selected (`window.getSelection().toString().length`)

- Screenshot taken **after** Ctrl+A (blue highlights in screenshot = selection enabled)

Decision logic (in strict priority order):

1. `bodyUserSelect == 'none'` → **PASS** (CSS blocks it at engine level — impossible to select)
2. `bodyUserSelect != 'none' AND keyboardSelectedLength == 0` → **PASS** (effectively blocked in practice)
3. `bodyUserSelect != 'none' AND keyboardSelectedLength > 0` → **FAIL** (text IS selectable)

Why pre-computed: This is pure measurement — there's no ambiguity in a CSS property value. The LLM confirms visually (blue highlights in screenshot) but the verdict is determined by the numbers.

CHECK 4 — Banner Load Time (`banner_Load_time`) → PerformanceAgent

What it checks: Did the banner fully load within 60 seconds?

Intuition: A banner that takes too long to load is never seen by the user. The 60-second limit is generous but absolute — if the banner's load event never fires or fires after 60s, it's a fail.

Evidence collected:

- `loadMs` — `window.performance.timing.loadEventEnd - navigationStart` (full page load in milliseconds)
- `domContentLoadedMs` — when the HTML parsed (before assets)
- `ttfbMs` — Time To First Byte (server response speed)
- `resourceCount` — how many resources (images, scripts) loaded
- `transferSizeKB` — total data downloaded
- `slowResources` — top 5 resources that took > 500ms to load

Grading:

- $\leq 3,000\text{ms}$ → excellent
- $\leq 10,000\text{ms}$ → good
- $\leq 30,000\text{ms}$ → acceptable

- $\leq 60,000\text{ms}$ $\rightarrow$ slow (still PASS)
- $60,000\text{ms}$ OR `null` $\rightarrow$ **FAIL**

Why null is a FAIL: If `loadEventEnd` is 0, the load event never fired — the banner is stuck/broken.

CHECK 5 — ISI Auto-Scroll (`isi_auto_scroll`) $\rightarrow$ ISIAgent

What it checks: Does the Important Safety Information (ISI) automatically scroll to reveal all content during the banner loop?

Intuition: ISI text (warnings, adverse reactions, prescribing info) is often hundreds of words long but the banner is only 250px tall. FDA regulation requires that ALL safety information be **conveyed** — so if the ISI overflows its visible area, the banner must auto-scroll through it as part of its animation loop.

Evidence collected:

- ISI element found via scoring algorithm (`#isi`, `.isi`, `[id*="isi"]`, GWD clip-box pattern)
- `scrollHeight` — how tall the ISI content actually is
- `clientHeight` — how tall the visible ISI window is
- `scrollablePixels` — `overflow = scrollHeight - clientHeight`
- `needsScroll` — `scrollablePixels > 5px`
- 5 position snapshots every 4 seconds ($T=0, 4, 8, 12, 16\text{s}$) — measures CSS transforms, `scrollTop`, GSAP animations
- `anyMovement` — did ANY snapshot differ from the previous by $> 1\text{px}$?

Decision logic:

1. `needsScroll = false` $\rightarrow$ **PASS** (ISI fits entirely — nothing to scroll)
2. `needsScroll = true` AND `anyMovement = true` $\rightarrow$ **PASS** (ISI scrolled automatically)
3. `needsScroll = true` AND `hasCustomScroll = true` AND `anyMovement = false` $\rightarrow$ **visual check** (GSAP transforms may not be measurable via DOM — compare screenshots)

4. needsScroll = true AND isNativeScroll = true AND anyMovement = false → **FAIL** (overflow exists, native scroller, but nothing moved)
5. needsScroll = true AND no scroller AND anyMovement = false → **FAIL**

GWD (Google Web Designer) pattern awareness: GWD banners use a clip-box where the outer `div#isi-area` has `overflow: hidden` and is only 67px tall, but a child `div.isi-narrow` is 1500px tall and moves up via CSS animation. Banner Automation Testing detects this by looking one level deeper into children.

CHECK 6 — ISI Text Layout (*isi_text_layout*) → *ISIAgent*

What it checks: Is the ISI text visually readable — no overlapping elements, no cut-off text?

Intuition: Even if the ISI scrolls correctly, it's useless if the text is overlapping (two paragraphs printed on top of each other) or cut off (text running outside its container). This can happen from CSS layout bugs.

Evidence collected:

- Overlap detection: scans all inline elements (`p`, `span`, `a`, `h1-h6`, `strong`, `em`, `li`, `td`, `th`) and checks if any two **sibling** elements have overlapping bounding boxes (both `xOverlap > 4px` AND `yOverlap > 4px`)
- `overlapCount` — number of overlapping pairs
- `pairs` — up to 6 overlap pairs with `elementA`, `elementB`, `xOverlap`, `yOverlap`, and text snippets
- Screenshot of ISI highlighted in blue outline

Pre-classification of overlaps (v10 fix):

Condition	Classification
Both inline (A, SPAN, STRONG) AND <code>yOverlap < 5px</code>	[artefact] — bounding box noise from line-height
Both inline AND <code>yOverlap ≥ 5px</code> AND <code>xOverlap > 30px</code>	[REAL] — two links sharing the same horizontal space
Any block element (P, DIV, H*) AND <code>yOverlap ≥ 5px</code>	[REAL] — paragraphs overlapping

Decision:

- `hasRealOverlaps = false` → look at screenshot → if text readable → **PASS**
- `hasRealOverlaps = true` → look at screenshot → if text visually overlapping → **FAIL**
- `checkedElements = 0` → ISI in cross-origin iframe → rely entirely on screenshot

CHECK 7 — ISI Wheel Scroll (*isi_wheel_scroll*) → *ISIAgent*

What it checks: Can a user manually scroll the ISI using their mouse wheel?

Intuition: Auto-scroll handles passive consumption, but an engaged user should also be able to scroll the ISI manually. More importantly, the wheel event must be **contained** — scrolling inside the ISI should NOT cause the whole page to scroll up/down (scroll leak).

Evidence collected:

- `positionBefore / positionAfter` — ISI scroll position before and after wheel event
- `deltaISI = positionAfter - positionBefore`
- `deltaWindow = window.scrollY` before/after (page scroll leak detection)
- `scrollLeaked = deltaWindow > 5px`
- Two screenshots: before and after wheel event

Decision logic:

1. `isScrollable = false` → **PASS** (ISI fits — no wheel needed, and page shouldn't scroll)
2. `isScrollable = true AND scrollLeaked = true` → **FAIL** (wheel escaped the ISI and scrolled the page)
3. `isScrollable = true AND deltaISI > 5px` → **PASS** (ISI responded to wheel correctly)
4. `isScrollable = true AND hasCustomScroll = true AND deltaISI ≤ 5px` → **visual check** (GSAP may not be measurable)
5. `isScrollable = true AND hasCustomScroll = false AND deltaISI ≤ 5px` → **FAIL** (ISI has overflow but wheel did nothing)

ISI Detection

Banner Automation Testing must find the ISI element across multiple browser frames and banner patterns. The scoring system:

ISI ELEMENT SCORING:

id contains "isi" keyword	→ +4 points
class contains "isi" keyword	→ +2 points
scrollablePixels > 100	→ +5 points
scrollablePixels > 0	→ +2 points
isNativeScroll = true	→ +3 points
textLength > 200 chars	→ +3 points
textLength > 50 chars	→ +1 point
height > 50px	→ +1 point

The highest-scoring candidate is chosen as the ISI element. This handles:

- Standard #isi selectors
- GWD clip-box pattern (#isi-area outer + #isi-narrow inner)
- Banners with ISI inside cross-origin iframes

Pharma Banner Detection

Before all ISI checks, Banner Automation Testing scans **all accessible frames** for pharma signals:

PHARMA SIGNALS (any match → isPharmaBanner = true):

```
"important safety information"
"prescribing information"
"indication"
"contraindication"
"warnings and precautions"
"adverse reaction"
"full prescribing"
```


"medication guide"
"see full"
"isi"

DRUG NAME PATTERN:

Regex: `/[A-Z]{4,}[®™]/` or `/[®™]\s*[A-Z]{3,}/`
Matches: NUBEQA®, LIVDELZI™, Nplate®

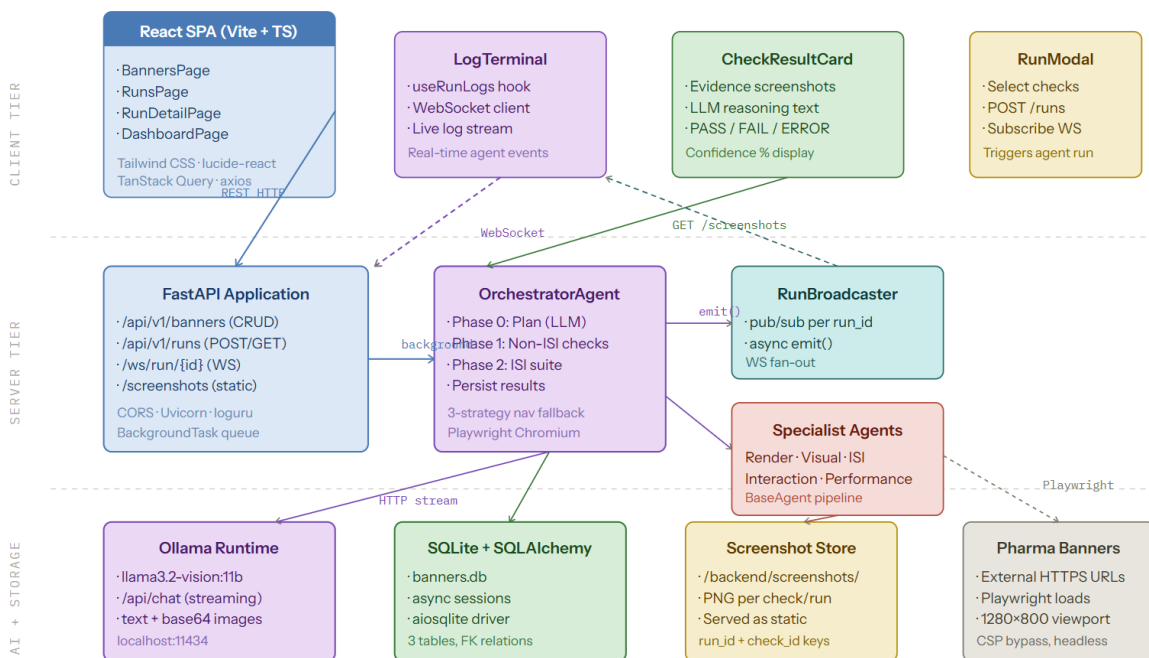
If `isPharmaBanner = true` AND `isiFound = false`:

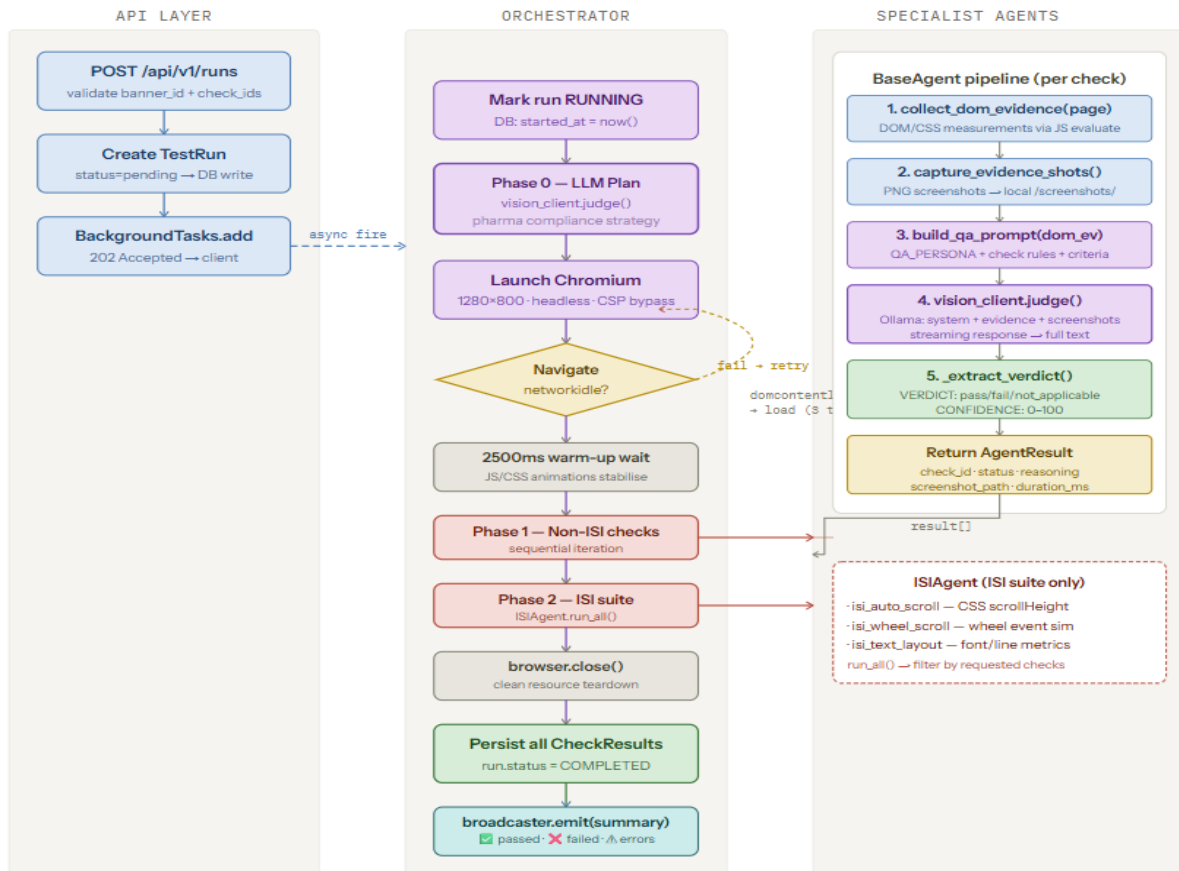
- The system overrides `not_applicable` → **FAIL**
- Rationale: a branded pharma ad **must** have ISI. Its absence is a critical compliance failure, not N/A.

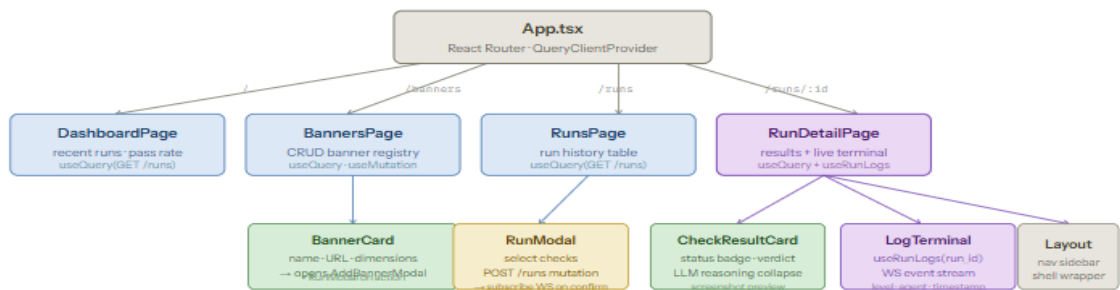
If `isPharmaBanner = false`:

- Non-branded awareness banner → ISI checks are `not_applicable`

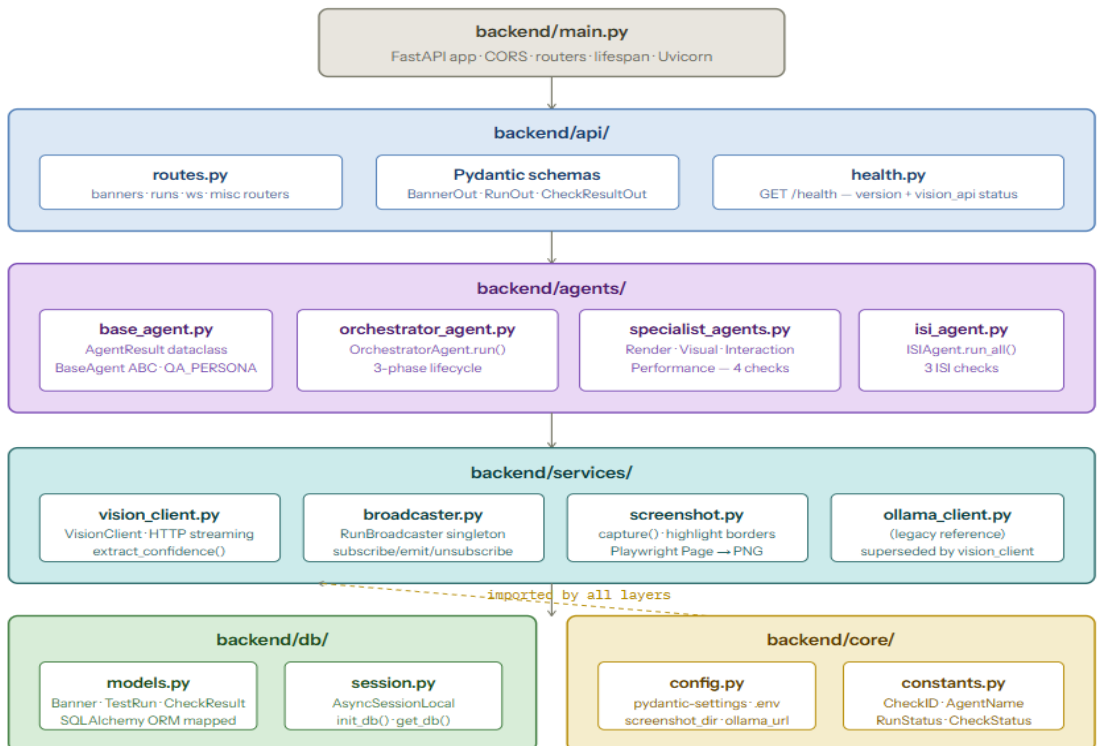
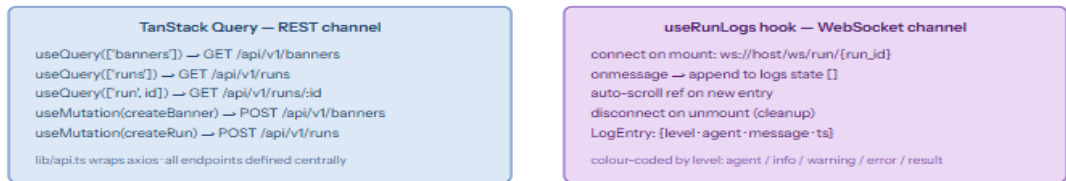
High-Level System Overview







DATA CHANNELS



PART 2: CODE EXECUTION

Full Request Lifecycle

User clicks "Run Tests" in UI



POST /api/v1/runs/ (FastAPI)



create_run() creates TestRun record in SQLite → returns 202 Accepted



BackgroundTasks.add_task(_fire_run, run_id, url, name, checks)

↓ (async background task)

OrchestratorAgent.run()



Phase 0: Plan text (LLM writes testing strategy)



Phase 1: Non-ISI checks (RenderAgent → VisualAgent → InteractionAgent → PerformanceAgent)



Phase 2: ISI suite (ISIAgent.run_all → auto_scroll → wheel_scroll → text_layout)



Persist all CheckResult rows to SQLite



Update TestRun: status=completed, counts



WebSocket broadcasts final summary to UI

main.py — Application Entry Point

```
python
# FastAPI app with lifespan context manager
@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()    # Creates SQLite tables on first startup
    yield              # App runs here
    # Cleanup on shutdown
```

Why lifespan: The modern FastAPI way to run startup/shutdown code. On first run, `init_db()` creates the `banners`, `test_runs`, and `check_results` tables via SQLAlchemy's `Base.metadata.create_all()`.

Static file serving:

```
python
app.mount("/screenshots",
StaticFiles(directory=str(settings.screenshot_dir)))
```

This serves captured screenshots at `/screenshots/filename.png` — the frontend uses these URLs to display evidence images inline.

core/config.py — Settings

Uses **Pydantic BaseSettings** which reads from environment variables or a `.env` file:

```
python
class Settings(BaseSettings):
    database_url: str = "sqlite+aiosqlite:///Banner Automation
```

Testing.db"

```
ollama_base_url: str = "http://localhost:11434"
ollama_vision_model: str = "llava:latest"
playwright_headless: bool = True
playwright_timeout: int = 60_000
screenshot_dir: Path = BASE_DIR / "screenshots"
```

Why SQLite: Zero-config, single file, perfectly adequate for a QA tool that isn't serving thousands of concurrent users. `aiosqlite` gives it async support.

Why Pydantic Settings: Type-safe configuration. Changing `PLAYWRIGHT_HEADLESS=false` in `.env` automatically type-casts to `bool`.

core/constants.py — Enums

python

```
class CheckID(str, Enum):
    BANNER_VISIBLE = "banner_visible"
    BORDER_DETECTED = "border_detected"
    COPY_SELECTION_DISABLED = "copy_selection_disabled"
    ISI_AUTO_SCROLL = "isi_auto_scroll"
    ISI_TEXT_LAYOUT = "isi_text_layout"
    ISI_WHEEL_SCROLL = "isi_wheel_scroll"
    BANNER_LOAD_TIME = "banner_load_time"
```

Why str, Enum: The enum values serialize directly to strings for JSON API responses and database storage without needing a custom serializer.

`CHECK_AGENT_MAP` maps each check to its responsible agent. `CHECK_LABELS` maps to human-readable display names for the UI.

db/models.py — Database Schema

Three SQLAlchemy ORM models:

Banner — the registry of banners to test:

id (UUID) | url_id (unique slug) | name | url | client | dimensions |
is_active

TestRun — each time someone runs a test:

id | banner_id (FK) | status | triggered_by | total_checks |
passed_checks | failed_checks | error_checks |
started_at | completed_at | orchestrator_reasoning

CheckResult — one row per check per run:

id | run_id (FK) | check_id | check_name | agent_name | status |
raw_data (JSON) | llm_reasoning (Text) | llm_verdict | final_verdict |
screenshot_path | duration_ms | error_message | executed_at

Why raw_data as JSON: The DOM evidence dict (scrollHeight, visibleElements, etc.) is stored as JSON so it can be displayed verbatim in the UI "Raw DOM evidence" section without any schema migrations if evidence fields change.

db/session.py — Database Connection

```
python
engine = create_async_engine(settings.database_url, echo=False)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db():
    async with AsyncSessionLocal() as session:
        yield session
```

Why expire_on_commit=False: SQLAlchemy by default expires all attributes after commit (to force a fresh DB read). In async code this causes MissingGreenlet errors. Disabling it lets objects be read after commit without re-querying.

api/routes.py — REST API

Banners:

- GET /api/v1/banners/ — list all banners
- POST /api/v1/banners/ — register a new banner URL
- PATCH /api/v1/banners/{id} — update banner metadata
- DELETE /api/v1/banners/{id} — remove from registry

Runs:

- POST /api/v1/runs/ — trigger a test run (returns 202 immediately, runs in background)
- GET /api/v1/runs/ — list runs (optional ?banner_id= filter)
- GET /api/v1/runs/{id} — get full run with all check results
- DELETE /api/v1/runs/{id} — delete a run

Why 202 Accepted for POST /runs/: A test run takes 2-5 minutes. Returning 200 would mean holding the HTTP connection open. Instead, the run is created in the DB immediately (so the frontend has an ID to poll/subscribe to) and the actual test executes in FastAPI's BackgroundTasks.

WebSocket:

python

```
@ws_router.websocket("/ws/runs/{run_id}")
async def run_ws(ws: WebSocket, run_id: str):
    broadcaster.subscribe(run_id, send)
    try:
        while True:
            await ws.receive_text()    # keep-alive loop
    except WebSocketDisconnect:
        broadcaster.unsubscribe(run_id, send)
```

Every time an agent logs a message, it calls `broadcaster.emit(run_id, message)` which pushes JSON to all WebSocket clients subscribed to that `run_id`. The frontend's `useRunLogs` hook connects to this and streams the live log terminal.

services/broadcaster.py — Real-time Pub/Sub

python


```

class RunBroadcaster:
    _subs: dict[str, set[Callable]]

    def subscribe(run_id, cb): # frontend connects → registers
callback
    def unsubscribe(run_id, cb): # frontend disconnects → removes
callback
    async def emit(run_id, message, level, agent):
        # Builds JSON payload, calls all registered callbacks

```

Why a custom broadcaster instead of Redis Pub/Sub: This is a local tool with one server. A simple in-memory dict of callbacks is sufficient and adds zero infrastructure complexity.

The JSON payload sent over WebSocket:

```

json
{
  "run_id": "abc-123",
  "level": "result",
  "agent": "isi",
  "message": "✓ ISI isi_auto_scroll: PASS (141s conf=100%)",
  "timestamp": "2025-05-20T10:30:00"
}

```

services/vision_client.py — Ollama LLM Client

```

python
class OllamaVisionClient:
    _CANDIDATE_MODELS = ["qwen2.5:7b", "llava:latest"]

    async def judge(system: str, text: str, screenshot_paths:
list[str]) -> str:
        model = await self._resolve_model() # auto-detect best
available model
        images = self._load_images(screenshot_paths) # load as base64
        # Build Ollama /api/chat payload
        # Stream response token by token (avoids read timeouts)
        return assembled_response_text

```

```

@staticmethod
def extract_confidence(text: str) -> int | None:
    # Parses "CONFIDENCE: 85" from LLM output
    m = re.search(r"CONFIDENCE\s*:\s*(\d{1,3})", text)
    return int(m.group(1)) if m else None

```

Why streaming: Ollama's `/api/chat` endpoint streams tokens one by one. If the response takes 90 seconds (large images + complex reasoning), a non-streaming call would timeout. Streaming collects tokens as they arrive and only returns when done: `true`.

Why base64 for images: Ollama's multimodal format requires images embedded as base64 strings in the message body. The client reads each PNG file and encodes it: `base64.standard_b64encode(raw).decode()`.

Model resolution: On first call, the client queries Ollama's `/api/tags` to see what's installed. It tries candidates in order (`qwen2.5:7b` → `llava:latest`) and uses whatever is available. Result is cached after first resolution.

Retry logic: 3 attempts with exponential backoff (2s, 4s) for transient Ollama errors.

agents/base_agent.py — BaseAgent

Every specialist agent extends `BaseAgent` and implements three abstract methods:

```

python
class BaseAgent(ABC):

    @abstractmethod
    async def collect_dom_evidence(self, page) -> dict:
        """Run JavaScript in the browser. Return raw measurements. No
        interpretation."""

    @abstractmethod
    async def capture_evidence_shots(self, page, run_id,
        screenshot_dir) -> list[str]:
        """Take screenshots. Return file paths."""

```

```

@abstractmethod
def build_qa_prompt(self, dom_ev: dict) -> tuple[str, str]:
    """Return (system_prompt, user_evidence_text) for the LLM."""

```

The run() method is the universal execution harness:

```

python
async def run(self, page, run_id, screenshot_dir) -> AgentResult:
    dom_ev = await self.collect_dom_evidence(page)          # Step 1
    shots = await self.capture_evidence_shots(...)          # Step 2
    system_prompt, user_evidence = self.build_qa_prompt(dom_ev) #
Step 3
    reasoning = await vision_client.judge(system_prompt,
user_evidence, shots) # Step 4
    verdict = self._extract_verdict(reasoning)              # Step 5
    confidence = vision_client.extract_confidence(reasoning)
    status = PASS if verdict in ("pass", "not_applicable") else FAIL
    return AgentResult(...)

```

_extract_verdict() — robust 3-level extraction:

1. Regex search for VERDICT: pass/fail/not_applicable anywhere in text
2. Scan last line for pass/fail keywords
3. Frequency count fallback (most mentioned word wins)

Why 3 levels: LLMs sometimes add markdown formatting, extra lines, or put the verdict in the middle. The 3-level extractor handles all of these gracefully.

agents/specialist_agents.py — Four Standard Agents

RenderAgent (CHECK-1: Banner Visible)

```

python
async def collect_dom_evidence(self, page) -> dict:
    return await page.evaluate("""() => {
        const visible =
Array.from(document.querySelectorAll('*')).filter(el => {
            const r = el.getBoundingClientRect(), s =

```

```

window.getComputedStyle(el);
    return r.width > 0 && r.height > 0
        && s.display !== 'none'
        && s.visibility !== 'hidden'
        && parseFloat(s.opacity) > 0;
    });
    ...
}""")

```

Why JavaScript `evaluate()`: DOM measurements can only be made in the browser's JavaScript context. Playwright's `page.evaluate()` executes JS inside the page and returns the result as a Python dict.

Why check opacity: CSS animations sometimes temporarily set `opacity: 0` on elements. We filter these out so visibility count is accurate at snapshot time.

VisualAgent (CHECK-2: Border Detected)

Takes **two screenshots**:

1. **Highlighted** — injects a red 4px solid red overlay on every detected border element, then screenshots
2. **Plain** — natural screenshot without modification

This gives the LLM both "where the system thinks the border is" (highlighted) and "what the user actually sees" (plain). It uses the `deterministicResult` pre-computed signal to avoid the LLM overthinking a clear CSS measurement.

InteractionAgent (CHECK-3: Copy Selection)

```

python
await page.keyboard.press("Control+a")
await asyncio.sleep(0.35)
css["keyboardSelectedLength"] = await page.evaluate(
    "() => window.getSelection()?.toString().length ?? 0"
)

```

Simulates a real user pressing Ctrl+A, waits for the browser to process the selection event, then measures how many characters are selected. Takes the screenshot after Ctrl+A so the LLM can literally see the blue text highlights.

PerformanceAgent (CHECK-4: Banner Load Time)

Uses the **Navigation Timing API** which is built into every browser:

```
javascript
window.performance.timing.loadEventEnd -
window.performance.timing.navigationStart
```

This is the gold standard for page load measurement — it's the same API Chrome DevTools uses.

agents/isi_agent.py — ISI Agent (Most Complex)

The ISI agent runs 3 checks in sequence, reloading the page between each to reset animation state.

Frame-safe evidence collection (_collect_banner_evidence)

```
python
for frame in page.frames:
    try:
        sig = await frame.evaluate(_PHARMA_SIGNAL_JS)    # scan for
drug names
    except Exception:
        frames_err += 1    # cross-origin frame – silently skip
    try:
        isi = await frame.evaluate(_ISI_FIND_JS)        # find ISI
element
    except Exception:
        pass
```

Why iterate all frames: Banners often load inside iframes (ad servers). The ISI element may be inside an inner <iframe> several levels deep. Playwright exposes all frames via `page.frames` and allows JavaScript execution in each one (unless cross-origin security blocks it).

ISI Element Detection (_ISI_FIND_JS)

JavaScript that runs inside the banner page to find the ISI element:

```
javascript
// 1. Find all div/section/aside elements matching ISI keywords
// 2. Score each candidate (scroll depth, text length, height, etc.)
// 3. Detect the GWD clip-box pattern (overflow:hidden outer + tall child)
// 4. Return the highest-scoring candidate with its metrics
```

GWD clip-box detection:

```
javascript
function getEffectiveScrollable(el) {
    // Check native scroll (overflow:scroll/auto)
    const ov = window.getComputedStyle(el).overflowY;
    if ((ov === 'scroll' || ov === 'auto') && el.scrollHeight >
el.clientHeight + 5) {
        return { isNative: true, scrollPx: el.scrollHeight -
el.clientHeight };
    }
    // GWD pattern: check if a CHILD is taller than the container
    for (const ch of Array.from(el.children)) {
        if (ch.getBoundingClientRect().height >
el.getBoundingClientRect().height + 20) {
            return { isNative: false, scrollPx: childHeight -
containerHeight, childSel };
        }
    }
}
```

Position reading(_read_pos_js)

Generates JavaScript that can read the ISI scroll position regardless of animation mechanism:

```
javascript
// Try 1: scrollTop (native scroll)
if (c.scrollTop > 1) return c.scrollTop;
```

```
// Try 2: CSS transform on child (GSAP animation)
const ts = window.getComputedStyle(ch).transform;
if (ts && ts !== 'none') {
    var m = new DOMMatrix(ts);
    if (Math.abs(m.m42) > 1) return -m.m42; // m42 is Y translation
}
// Try 3: inline style top
var top = parseFloat(ch.style.top || '0');
if (!isNaN(top) && Math.abs(top) > 1) return -top;
```

Why negative values: When a div moves upward (as ISI content scrolls up to reveal new lines), its translateY is negative (e.g., -200px). Negating it gives a positive "how much has scrolled" reading consistent with scrollTop.

Auto-Scroll check with pre-computed verdict (v10)

```
python
# Python decides the verdict BEFORE calling the LLM
if not needs_scroll:
    pre_verdict = "pass"
    pre_reason = "ISI fits in container – no scroll required"
elif has_movement:
    pre_verdict = "pass"
    pre_reason = f"ISI scrolled {total_movement}px over 16s"
elif has_custom:
    pre_verdict = "visual" # must compare screenshots
    pre_reason = "hasCustomScroll=true – compare screenshots"
elif is_native:
    pre_verdict = "fail"
    pre_reason = "overflow but no movement detected"

# Prompt tells LLM the pre-computed verdict explicitly
verdict_instruction = (
    f"PRE-COMPUTED VERDICT: {pre_verdict.upper()}\n"
    f"REASON: {pre_reason}\n\n"
    f"Confirm by examining screenshots, then output VERDICT: {pre_verdict}."
)
```

Why pre-computed verdict instead of decision tree: The original approach gave the LLM a 4-step decision tree and hoped it would traverse it correctly. Tests showed the LLM would sometimes get confused in step 3 and make wrong choices. Sending the answer with reasoning and asking the LLM to confirm is faster, more accurate, and retains the LLM's visual confirmation value.

Pharma override (v10)

python

```
if verdict == "not_applicable":
    pharma = dom_ev.get("pharma", {})
    isi_found = dom_ev.get("isiFound", True)
    if pharma.get("isPharmaBanner") and not isi_found:
        verdict = "fail"    # Override: pharma banner MUST have ISI
```

agents/orchestrator_agent.py — Coordinator

python

```
class OrchestratorAgent:
    async def run(self, run_id, banner_url, banner_name, check_ids):
        # Phase 0: Planning
        plan = await vision_client.judge(PLAN_SYSTEM, prompt, [])

        # Phase 1: Open Playwright browser
        async with async_playwright() as pw:
            browser = await pw.chromium.launch(headless=True,
args=[...])
            page = await ctx.new_page()

            # Navigation with 3-strategy fallback
            for strategy in ("networkidle", "domcontentloaded",
"load"):
                try:
                    await page.goto(banner_url, wait_until=strategy)
                    break
                except Exception:
                    continue
```



```

        await page.wait_for_timeout(2500) # warm-up for JS
animations

        # Phase 1: Standard checks
        for cid in non_isi_checks:
            results.append(await agent_cls().run(page, run_id,
shot_dir))

        # Phase 2: ISI suite
        all_isi = await ISIAgent().run_all(page, run_id, shot_dir)

```

Why 3 navigation strategies:

1. networkidle — waits until no network activity for 500ms. Best for animated banners that load all assets. Sometimes times out on banners that keep polling.
2. domcontentloaded — HTML parsed, no waiting for images. Faster fallback.
3. load — fire-and-forget load event.

Why 2500ms warm-up: JavaScript animation frameworks (GSAP, TweenLite, GWD) initialize asynchronously after load. Without a warm-up, the ISI scroll animation may not have started yet when the agent takes its first snapshot.

Browser launch args:

```

python
"--no-sandbox"                # required in Docker/CI
"--disable-dev-shm-usage"     # prevents crashes on low-memory
hosts
"--ignore-certificate-errors" # banner CDNs often use internal
certs
"--disable-web-security"      # allows cross-origin frame access
"--allow-running-insecure-content"# HTTP assets on HTTPS pages

```

services/screenshot.py — Screenshot Helpers

Three capture modes:

capture(page, path) — standard full-page screenshot

capture_with_border_highlight(page, path) — injects CSS to draw red boxes on border elements:

```
javascript
// Inject a <style> that outlines border-having elements
document.querySelectorAll('*').forEach(e1 => {
  const s = getComputedStyle(e1);
  if (parseFloat(s.borderTopWidth) > 0) {
    e1.style.outline = '4px solid red';
  }
});
```

capture_isi_highlighted(page, sel, path) — injects blue outline on the ISI element:

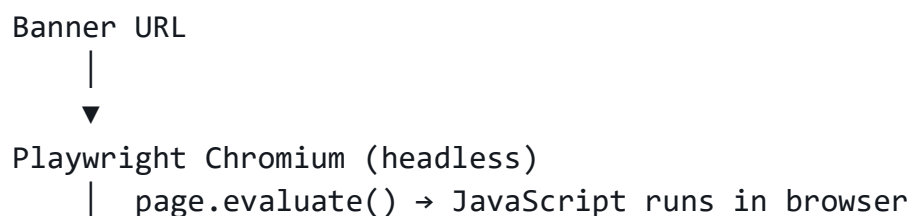
```
javascript
const e1 = document.querySelector('#isi');
e1.style.outline = '3px solid blue';
e1.style.outlineOffset = '2px';
```

This makes the evidence screenshots self-explanatory for the LLM — the LLM doesn't need to guess where the border or ISI is; it's visually marked.

Each agent follows the same 5-step pattern:

1. **Collect** — JavaScript in browser → Python dict
2. **Capture** — screenshots to disk
3. **Build** — system prompt + evidence text
4. **Judge** — send to Ollama → get reasoning
5. **Extract** — parse VERDICT from reasoning

Data Flow Diagram





Frontend Architecture

```
src/
├─ main.tsx          # React app entry, QueryClient setup
├─ App.tsx           # Router setup with 4 pages
├─ pages/
│   ├─ DashboardPage.tsx # Overview: stats, recent runs table
│   ├─ BannersPage.tsx   # Banner registry: add/list/run
│   ├─ RunsPage.tsx      # List of all test runs
│   └─ RunDetailPage.tsx # Live run view: log + results grid
├─ components/
│   └─ ui/              # Shared: StatusBadge, AgentBadge, Spinner,
etc.
    ├─ banners/         # Banner-specific components
    ├─ runs/            # LogTerminal (live WebSocket log)
    └─ results/
        └─ CheckResultCard.tsx # Per-check result with screenshots
├─ hooks/
│   └─ useRunLogs.ts     # WebSocket hook → live log entries
├─ lib/
│   └─ api.ts            # Axios HTTP functions (getBanners, getRun,
etc.)
└─ types/
    └─ index.ts          # TypeScript interfaces
```

TanStack Query polling for live runs:

```
typescript
const { data: run } = useQuery({
  queryKey: ['run', runId],
  queryFn: () => getRun(runId!),
  refetchInterval: q =>
    q.state.data?.status === 'running' ? 2500 : false,
})
```

While a run is active, the frontend polls every 2.5 seconds AND listens on WebSocket simultaneously. The poll ensures the results grid updates even if a WebSocket message was missed.

useRunLogs hook:

typescript

```
export function useRunLogs(runId: string | null) {
  const [logs, setLogs] = useState<LogEntry[]>([])
  useEffect(() => {
    const ws = new
WebSocket(`ws://localhost:8000/ws/runs/${runId}`)
    ws.onmessage = (e) => {
      const entry = JSON.parse(e.data)
      setLogs(prev => [...prev, entry])
    }
    return () => ws.close()
  }, [runId])
  return logs
}
```

Execution Phases Detail

Phase 0 — Planning (0-5 seconds)

The Orchestrator calls the LLM with:

System: "You are QA lead for pharma banner compliance testing..."

User: "Banner: Nubeqa Safety 300x250

URL: <https://lms-us.indegene.com/.../300x250/index.html>

Checks: [banner_visible, border_detected,
copy_selection_disabled,
banner_load_time, isi_auto_scroll, isi_text_layout,
isi_wheel_scroll]"

The LLM generates a 2-3 sentence testing strategy (e.g., "This is a branded pharma banner — ISI compliance is the primary risk..."). This is stored as `orchestrator_reasoning` and displayed in the UI.

Phase 1 — Standard Checks (30-120 seconds)

Four agents run sequentially on the same Playwright page:

- RenderAgent (~60-110s for LLM)
- VisualAgent (~80-110s — two screenshots to process)
- InteractionAgent (~60s)
- PerformanceAgent (~60s)

Phase 2 — ISI Suite (300-500 seconds)

The ISI suite reloads the page before each check:

- Auto-scroll: 16s of position snapshots + 2×LLM calls (~140s total)
- Wheel scroll: wheel event + 2.5s wait + 2×LLM calls (~170s)
- Text layout: overlap detection + 1×LLM call (~80s)

Total run time for 7 checks: ~10-15 minutes

Why These Specific Technology Choices

Technology	Choice	Why
Browser	Playwright Chromium	Full Chrome engine, CSS computed styles, JS evaluation, multi-frame, screenshot API
Vision LLM	Ollama (local)	Zero API cost, no data leaving network, works offline, HIPAA-friendly for pharma
Vision model	llava / llama3.2-vision	Best open-source models for reading text in screenshots
Backend	FastAPI async	Native asyncio + Playwright async API + SQLAlchemy async — no thread pool hacks
Database	SQLite	Zero infrastructure for a single-machine QA tool; aiosqlite for async
Frontend state	TanStack Query	Built-in polling, stale-while-revalidate, automatic refetch on window focus
Real-time	WebSocket	Native FastAPI support, no broker needed, live log streaming
Screenshot format	PNG	Lossless, LLMs read PNG better than JPEG for text content

Key Design Decisions

1. Pure Agentic, No Deterministic Fallback The system never hard-codes pass/fail. All verdicts come from the LLM reading evidence. The pre-computed signals are *inputs to the*

prompt, not bypass switches. This means the LLM can catch edge cases a rule-based system would miss — but it also means the prompts must be crystal-clear.

2. Evidence-First Prompting Every prompt follows: *collect facts* → *pre-classify* → *show LLM the answer with reasoning* → *ask to confirm*. This is more reliable than: *show LLM facts* → *ask it to decide*. The LLM's value is in visual verification and handling edge cases, not in basic math.

3. Screenshot as Primary Evidence The LLM always receives a screenshot along with DOM data. This lets it catch cases where CSS numbers say one thing but the rendered output looks different (rendering bugs, z-index issues, etc.).

4. Page Reload Between ISI Checks Auto-scroll, wheel scroll, and text layout all require the banner to be in a fresh state. Reloading ensures the animation hasn't finished, scroll position is reset, and any state from the previous check doesn't contaminate the next one.

5. Multi-Frame Scanning Most pharma banner failures happen because the ISI lives inside a nested iframe that naive CSS selectors miss. Banner Automation Testing explicitly iterates `page.frames` and runs detection JS in each accessible frame, giving it a major accuracy advantage over single-frame scanners.

Folder Structure Summary

```
Banner Automation Testing_v10_vision_updated/
├── backend/
│   ├── main.py                # FastAPI app, CORS, static files
│   ├── requirements.txt       # Python dependencies
│   └── core/
│       ├── config.py          # Pydantic Settings (.env support)
│       └── constants.py       # Enums: CheckID, AgentName,
CheckStatus
│   ├── db/
│   │   └── models.py          # SQLAlchemy ORM: Banner, TestRun,
CheckResult
│   │   └── session.py         # Async engine, session factory
│   ├── api/
│   │   └── routes.py          # All REST endpoints + WebSocket
│   └── agents/
```

```

|   |   |─ base_agent.py           # BaseAgent ABC + QA_PERSONA +
AgentResult
|   |   |─ specialist_agents.py    # RenderAgent, VisualAgent,
InteractionAgent, PerformanceAgent
|   |   |─ isi_agent.py             # ISIAgent (auto_scroll,
wheel_scroll, text_layout)
|   |   └─ orchestrator_agent.py    # OrchestratorAgent: coordinates
the full run
|   └─ services/
|       └─ vision_client.py         # Ollama HTTP client (streaming,
retry, base64 images)
|       └─ screenshot.py           # Playwright screenshot helpers
(plain, highlighted, ISI)
|       └─ broadcaster.py          # WebSocket pub/sub for live log
streaming
└─ frontend/
    └─ src/
        └─ App.tsx                 # Router
        └─ pages/                  # Dashboard, Banners, Runs,
RunDetail
|   |   |─ components/             # UI components
|   |   └─ results/CheckResultCard.tsx # Per-check result card
with auto-expand on fail
|   |   |─ hooks/useRunLogs.ts     # WebSocket live log hook
|   |   |─ lib/api.ts              # HTTP functions
|   |   └─ types/index.ts          # TypeScript interfaces
|   └─ package.json                # React, Vite, TailwindCSS,
TanStack Query
|   └─ vite.config.ts              # Dev server proxy to backend :8000
└─ scripts/
    └─ seed.py                     # Seed script: insert sample
banners into DB

```